

Today 4:53 PM

I am currently designing a logistic regression for a dataset containing text, numbers, and "unknown" values but the model only reads numbers. Give me a lesson on the methods and libraries needed to develop the regression model and how to differentiate between using one hot encoding and converting the data column into binary values. Develop code from an example dataset and break down the lesson into the pre-processing, modeling, training, and testing steps. The testing should involve the use of the following metrics: ROC_AUC, recall, precision, accuracy, F1

Worked for 6s >

Logistic regression with mixed data

Logistic regression predicts the probability of a **binary outcome**, such as whether a wine is high quality (1) or not (0). It can work with text, numbers, and missing values, but those inputs must be converted into numeric features first.

This example uses `pandas` to organize the data and `scikit-learn` for preprocessing, logistic regression, and evaluation. The preprocessing steps are placed in a **pipeline** so the same transformations are learned from the training set and then applied to the test set.

1. Example dataset

Imagine predicting whether a customer renews a subscription. The dataset has numeric, categorical, and binary columns; some categorical entries are "unknown" .

age	monthly_spend	plan	contacted	renewed
22	35.0	basic	no	0
41	80.0	premium	yes	1
35	55.0	unknown	no	0
29	60.0	standard	yes	1

- `renewed` is the target, or label.

- `age` and `monthly_spend` are numeric features.
- `plan` is a categorical feature with several possible values.
- `contacted` is a binary feature with two possible values.
- `"unknown"` is a value in the data. Decide whether it means **missing information** or a meaningful category before encoding it.

2. Preprocessing: encode and scale the features

One-hot encoding

Use one-hot encoding when a column has **three or more unordered categories**, such as `plan`. It creates a separate 0/1 column for each category:

plan	plan_basic	plan_premium	plan_standard
basic	1	0	0
premium	0	1	0
standard	0	0	1

Do not convert categories like `basic`, `standard`, and `premium` to `1`, `2`, and `3` unless their numeric ordering and distances have real meaning. Otherwise, the model could interpret the numbers as a meaningful scale.

Binary encoding

For a column with exactly two categories, such as `contacted`, map the values to 0 and 1:

</> Plain text

```
no    -> 0
yes   -> 1
```

One-hot encoding would also work for two categories, but a single binary column is simpler. For a two-category variable, the choice mostly affects representation, not the information available to the model.

What to do with `"unknown"`

There are two common cases:

1. **It means information is missing.** Convert it to a missing value and impute it. For categorical columns, a common choice is to replace missing values with the most frequent training-set

category. You can also add a missingness indicator if the fact that a value is missing may itself be predictive.

2. **It is a meaningful category.** Keep "unknown" as a category. One-hot encoding will create an unknown indicator column.

Do not automatically remove rows containing "unknown" : that discards data and can bias the model if those rows differ from the rest.

3. Modeling and training

Here is a runnable example. The dataset is deliberately small to demonstrate the workflow; real model evaluation requires substantially more data.

</> Python

```
import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Example data
df = pd.DataFrame({
    "age": [22, 41, 35, 29, 51, 46, 24, 38, 33, 57,
           31, 44, 27, 49, 36, 54, 25, 42, 32, 47],
```

StandardScaler puts numeric features on comparable scales, which can help logistic regression fit reliably. class_weight="balanced" adjusts for unequal class frequencies; it is a useful option to evaluate when one outcome is much rarer, but it is not always the best choice.

If your scikit-learn version does not accept sparse_output=False , replace it with sparse=False .

4. Testing and evaluation

The model returns probabilities as well as predicted classes. The default class threshold is 0.5. These metrics use that default threshold except for ROC AUC, which evaluates how well the model ranks positive cases above negative ones across thresholds.

</> Python

```
# Predicted probability of class 1, and predicted class labels.
y_probability = model.predict_proba(X_test)[: , 1]
y_prediction = model.predict(X_test)

metrics = {
    "ROC_AUC": roc_auc_score(y_test, y_probability),
    "recall": recall_score(y_test, y_prediction, zero_division=0),
    "precision": precision_score(y_test, y_prediction, zero_division=0),
    "accuracy": accuracy_score(y_test, y_prediction),
    "F1": f1_score(y_test, y_prediction, zero_division=0),
}

for name, value in metrics.items():
    print(f"{name}: {value:.3f}")
```

- **ROC AUC:** How well the model ranks positive cases above negative cases across possible thresholds. It uses predicted probabilities, not class labels.
- **Recall:** Of all actual positive cases, what fraction did the model find?
- **Precision:** Of the cases predicted positive, what fraction were actually positive?
- **Accuracy:** What fraction of all predictions were correct? It can be misleading when one class is much more common than the other.
- **F1:** The harmonic mean of precision and recall. It is useful when you want a single score that balances those two metrics.

Choose the threshold based on the cost of mistakes. For example, if missing a positive case is especially costly, a lower threshold may improve recall, though it can reduce precision. ROC AUC does not choose that threshold for you.

5. Applying this to your dataset

Before fitting the model, make a simple feature plan:

Column type	Example	Typical treatment
Numeric	age, price, count	Impute missing values; often scale
Two categories	yes/no, true/false	Map to 0/1 or use one-hot encoding

Column type	Example	Typical treatment
Multiple unordered categories	city, product type	One-hot encode
"unknown" means missing	response not recorded	Convert to missing, then impute; consider a missingness indicator
"unknown" is meaningful	respondent selected "unknown"	Keep as a category and encode

Finally, use a larger holdout set or cross-validation for dependable evaluation. With only 20 rows in the example, the displayed test scores would be highly sensitive to which rows happened to land in the test split.

what is the syntax for removing "unknown" from every column in the data set? use the name "x" for the data

To remove every row that contains the exact value "unknown" in any column of a pandas DataFrame named `x` :

</> Python

```
x = x[~x.eq("unknown").any(axis=1)]
```

`x.eq("unknown")` checks every cell, `.any(axis=1)` finds rows with at least one match, and `~` keeps the rows without a match.

is `x = x.replace("unknown", np.nan)` considered a valid method for replacing all the unknown values?

Yes. If `x` is a pandas DataFrame, this replaces every cell containing the exact string "unknown" with NaN :

</> Python

```
x = x.replace("unknown", np.nan)
```

Make sure you've imported NumPy:

```
</> Python
```

```
import numpy as np
```

This marks those values as missing; it does not remove the rows.

explain each of the methods used in this segment of code, the reason for the variable name given, and what stage in the logistic regression model this section contributes to: `numeric_pipeline =`

```
Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])
```

```
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])
```

```
binary_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(
        drop="if_binary",
        handle_unknown="ignore",
        sparse_output=False,
    )),
])
```

These three **preprocessing pipelines** handle different kinds of input columns before the values reach logistic regression. A `Pipeline` applies its listed steps in order.

Numeric pipeline

```
</> Python
```

```
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])
```

- **numeric_pipeline** is a descriptive variable name: this pipeline is intended for numeric columns.
- **Pipeline(...)** chains preprocessing steps together.
- **("imputer", ...)** names the step "imputer" . `SimpleImputer(strategy="median")` fills missing numeric values with that column's median, calculated from the training data.
- **("scaler", ...)** names the next step "scaler" . `StandardScaler()` centers numeric values around 0 and scales them so they have a standard deviation of about 1. This helps keep features with large numeric ranges from dominating the fit.

Stage: Numeric-feature preprocessing.

Categorical pipeline

</> Python

```
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])
```

- **categorical_pipeline** indicates that this pipeline is for categorical columns, such as "plan" with values like "basic" or "premium" .
- **("imputer", ...)** fills missing values with the most frequent category in that column, based on the training data.
- **("onehot", ...)** names the encoding step. `OneHotEncoder` turns each category into a numeric indicator column. For example, `plan_basic` is 1 when the plan is basic and 0 otherwise.
- **handle_unknown="ignore"** tells the encoder what to do if a category appears at prediction time that it did not see during training. It ignores that category instead of raising an error.

Stage: Categorical-feature preprocessing.

Binary pipeline

</> Python

```
binary_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(
        drop="if_binary",
        handle_unknown="ignore",
        sparse_output=False,
    )),
])
```

- **binary_pipeline** identifies this as the pipeline for columns with two possible categories, such as "yes" and "no" .
- The **imputer** fills missing values with the most frequent category.
- **OneHotEncoder(...)** converts the categories to numbers. `drop="if_binary"` drops one of the two indicator columns, so the feature is represented with one column instead of two. For example, "yes" and "no" can be represented with a single 0/1 column.
- **handle_unknown="ignore"** prevents an error if an unseen category occurs later.
- **sparse_output=False** asks the encoder to return a regular dense array instead of a sparse matrix.

Stage: Binary-feature preprocessing.

What do the quoted names mean?

Names like "imputer" , "scaler" , and "onehot" are labels for the steps inside each pipeline. They make the steps easier to identify and refer to later; they do **not** determine how the step works. The variable names `numeric_pipeline` , `categorical_pipeline` , and `binary_pipeline` are also chosen for clarity—you could use different valid Python names.

These pipelines prepare features; they do **not** train logistic regression by themselves. Typically, they are connected to selected DataFrame columns with a `ColumnTransformer` , then combined with a logistic regression estimator in an overall pipeline.

provide feedback on the logistic regression model I developed for my dataset. assume that the data file was imported with the variable "data" representing the call to that file. do not make any changes to the code but provide suggestions on making the regression model more accurate by identifying the line of code then explaining the advice

```
print(data.head())
print(data.dtypes)
```

```
x = data.drop(columns = ["row_id","y","duration"])
x_with_duration = data.drop(columns = "y")
y = data["y"].map({"no":0, "yes":1})
```

#Ignore 999 in days, drop unknown in loans,

```
textColumns = x.select_dtypes(include = ["object"]).columns
x = pd.get_dummies(x,columns = textColumns, dtype =int)
```

```
x_train, x_test, y_train, y_test= train_test_split(
    x,y,test_size=0.2,random_state=50, stratify = y
)
```

```
model = LogisticRegression(max_iter = 1000)
model.fit(x_train,y_train)
```

```
probability_yes = model.predict_proba(x_test)[:,-1]
```

```
predictions = []
```

```
for p in probability_yes:
    if p >= 0.33:
        predictions.append(1)
    else:
        predictions.append(0)
print("Accuracy", accuracy_score(y_test,predictions))
print("Precision",
precision_score(y_test,predictions,zero_division=0))
print("Recall", recall_score(y_test, predictions,zero_division=0))
print("F1", f1_score(y_test, predictions,zero_division=0))
print("ROC_AUC", roc_auc_score(y_test,probability_yes))
```

You have the main workflow in place: define features and target, split the data, fit logistic regression, predict probabilities, and evaluate several metrics. Here are suggestions tied to specific lines. I'm assuming `y` is a binary "no" / "yes" outcome and that `duration` may be measured after or during the event being predicted.

Data preparation

Line:

```
</> Python
```

```
x = data.drop(columns = ["row_id","y","duration"])
```

Dropping `row_id` is usually sensible because an identifier rarely represents a meaningful predictive signal. Excluding `y` prevents target leakage into the features. Excluding `duration` may also be important: if it is only known after an event, using it would make the model's results look better than what would be possible when making a real prediction. Keep it excluded if the goal is to predict *before* the duration is known.

You also create `x_with_duration` :

```
</> Python
```

```
x_with_duration = data.drop(columns = "y")
```

That variable is not used in the code shown. If you later evaluate a version that includes `duration` , make sure that comparison reflects when the model would actually be used. A feature available only after the outcome should not be used for an early prediction.

Lines:

```
</> Python
```

```
#Ignore 999 in days, drop unknown in loans,
```

This comment doesn't implement either operation. Decide what `999` and `"unknown"` mean in the source data before handling them. If `999` means "not applicable" or "not recorded," it should generally be treated as missing, not as an actual number. For `"unknown"` in `loans` , consider whether it means missing information or is itself a meaningful category. Dropping rows with unknown values may reduce your sample and can bias results if those rows differ systematically from other records.

Lines:

```
</> Python
```

```
textColumns = x.select_dtypes(include = ["object"]).columns  
x = pd.get_dummies(x, columns = textColumns, dtype = int)
```

This one-hot encodes all object-type columns, which is a reasonable baseline for categorical data. Check that every object column is actually categorical; free-form text needs different treatment, and numeric values accidentally stored as text may need conversion.

Pandas creates dummy columns for the categories it sees in all of `x` before the split. For a stricter evaluation, fit the encoding using only training data and apply the same transformation to the test data. A scikit-learn `ColumnTransformer` with `OneHotEncoder(handle_unknown="ignore")` can do this within a pipeline and safely handle categories that appear only in the test or future data.

Split and model fitting

Lines:

</> Python

```
x_train, x_test, y_train, y_test= train_test_split(  
    x,y,test_size=0.2,random_state=50, stratify = y  
)
```

Using `stratify=y` is helpful when the "yes" class is less common because it keeps the class proportions similar in training and test sets. A single split can still give unstable results, especially with a small dataset. For a more reliable estimate, use stratified cross-validation on the training data, then reserve the test set for a final evaluation.

Lines:

</> Python

```
model = LogisticRegression(max_iter = 1000)  
model.fit(x_train,y_train)
```

`max_iter=1000` gives the solver more iterations to converge. Also consider scaling numeric features: logistic regression often benefits when numeric columns are on comparable scales. Scaling should be fitted on training data only, typically through a pipeline.

You can also tune regularization, such as `C`, and compare suitable solvers using cross-validation. The default regularization may not be optimal for your dataset. If categories are very rare, regularization can help limit overly large coefficients.

Threshold and metrics

Lines:

```
</> Python
```

```
if p >= 0.33:
```

A 0.33 threshold predicts "yes" more readily than the standard 0.5 threshold. That often raises recall while lowering precision, but whether it is better depends on the costs of false negatives and false positives. Choose the threshold using cross-validated predictions on the training data, based on the metric or tradeoff you care about. Then evaluate the chosen threshold once on the untouched test set.

Lines:

```
</> Python
```

```
print("Accuracy", accuracy_score(y_test,predictions))
print("Precision", precision_score(y_test,predictions,zero_division=0))
print("Recall", recall_score(y_test, predictions,zero_division=0))
print("F1", f1_score(y_test, predictions,zero_division=0))
print("ROC_AUC", roc_auc_score(y_test,probability_yes))
```

These metrics give a useful mix:

- **Accuracy** is the share of correct predictions, but can look high when one class dominates.
- **Precision** measures how many predicted "yes" cases really are "yes" .
- **Recall** measures how many actual "yes" cases the model finds.
- **F1** balances precision and recall at your chosen threshold.
- **ROC AUC** measures how well the predicted probabilities rank positive cases above negative cases across thresholds. It correctly uses `probability_yes` , rather than the thresholded predictions.

For an imbalanced outcome, compare these metrics with the proportion of "yes" cases and consider a precision–recall curve as well. The best model is not necessarily the one with the highest accuracy; it depends on which errors matter most.

Most useful next steps: clarify the meaning of 999 and "unknown" , ensure the features are available at prediction time, put encoding and scaling inside a training pipeline, and select the probability threshold through cross-validation rather than choosing it by inspecting test-set results.

how do you measure the effectiveness of a logistic regression model? Does my model reflect the characteristics of a successful regression model

Worked for 4s >

A logistic regression model is a **classification model** when it predicts a binary outcome such as "yes" or "no". Its effectiveness means more than getting a high score: it should perform well on new data, and its errors should fit the cost of mistakes in your use case.

What the metrics tell you

Your code calculates five useful metrics:

- **Accuracy:** What fraction of all predictions were correct? It can be misleading if "yes" is rare; a model that always predicts "no" might still score well.
- **Precision:** Of the cases predicted "yes", what fraction really were "yes" ?
- **Recall:** Of all actual "yes" cases, what fraction did the model find?
- **F1:** A combined score that balances precision and recall.
- **ROC AUC:** How well the model ranks "yes" cases above "no" cases across probability thresholds. It evaluates probabilities, so using `probability_yes` here is appropriate.

No one metric establishes success by itself. For instance, if missing a "yes" case is costly, recall may matter more; if false alarms are costly, precision may matter more. You should compare the model to a simple baseline, such as always predicting the more common class, and check whether its scores improve on that baseline.

Does your model have good practices?

Yes, it has several important pieces in place:

- You separate the features from the target and drop `row_id`, which is generally an identifier rather than a useful predictor.
- `stratify=y` helps keep the proportions of "yes" and "no" similar in both sets.
- You evaluate on a held-out test set instead of only reporting training performance.
- You use predicted probabilities for ROC AUC and thresholded predictions for the other metrics, which is the right distinction.
- `zero_division=0` lets precision or recall be calculated without an exception if a split has no predicted positive cases.

That's a solid starting structure. But without the actual metric values, dataset size, and "yes" rate, I can't tell whether the model performs well in practice.

What could make the evaluation more reliable

1. Choose the threshold deliberately.

Your threshold is `0.33`. That predicts "yes" more readily than the usual `0.5`, which may increase recall and reduce precision. This can be a sensible choice, but base it on the tradeoff you need. Choose it using training data or cross-validation, then evaluate the chosen threshold once on the test set. Do not repeatedly adjust it based on test results.

2. Check class balance and the confusion matrix.

Look at the number of "yes" and "no" examples, then inspect true positives, false positives, true negatives, and false negatives. The confusion matrix helps explain *which* mistakes drive your precision, recall, and accuracy scores.

3. Use cross-validation during model development.

One random train/test split can give a score that depends heavily on which rows landed in each set. Stratified cross-validation gives you a more stable view of performance across multiple splits. Keep the test set for the final check. Scikit-learn supports cross-validation and multiple classification metrics for this purpose.

4. Keep preprocessing within the split.

Your one-hot encoding is done before `train_test_split`. For a more rigorous evaluation, fit encoding and any imputation or scaling only on training folds, then apply those learned transformations to validation or test data. A scikit-learn `Pipeline` and `ColumnTransformer` help prevent information from leaking across the split.

5. Confirm the predictors are available when predictions will be made.

Excluding `duration` may be appropriate if duration is only known after the event you're predicting. More generally, any feature recorded after the outcome can make test scores look better than real-world performance.

6. Review probability quality if you use probabilities.

ROC AUC assesses ranking, not whether a predicted probability such as `0.8` corresponds to an 80% event rate. If probabilities themselves matter, also examine log loss or a calibration plot.

So, your model reflects several characteristics of a good evaluation workflow, but its **effectiveness** depends on the resulting scores, the "yes" class frequency, the error costs, and whether the test data represents the situations where you'll use it.

What lines of my code need to be changed and what needs to be added to keep preprocessing within the split??

The key change is to split the **raw features first**, then fit encoding on the training data and use that same encoder to transform the test data. Replace your current `get_dummies` line and split section

with this:

</> Python

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder

# Define features and target from the original data
x = data.drop(columns=["row_id", "y", "duration"])
y = data["y"].map({"no": 0, "yes": 1})

# Split before fitting the encoder
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=50, stratify=y
)

# Identify categorical columns using training data
textColumns = x_train.select_dtypes(include=["object"]).columns

# Fit the encoder only on training data; then transform both sets
encoder = OneHotEncoder(handle_unknown="ignore", sparse_output=False)

x_train_encoded = encoder.fit_transform(x_train[textColumns])
x_test_encoded = encoder.transform(x_test[textColumns])
```

That handles the categorical columns, but you also need to retain the numeric columns and combine them with the encoded ones:

</> Python

```
numericColumns = x_train.select_dtypes(exclude=["object"]).columns

x_train_final = np.concatenate(
    [x_train[numericColumns].to_numpy(), x_train_encoded], axis=1
)
x_test_final = np.concatenate(
    [x_test[numericColumns].to_numpy(), x_test_encoded], axis=1
)
```

Then fit and predict using the new feature variables:

</> Python

```
model = LogisticRegression(max_iter=1000)
model.fit(x_train_final, y_train)
```

```
probability_yes = model.predict_proba(x_test_final)[:, 1]
```

You'll need these imports:

</> Python

```
import numpy as np
from sklearn.preprocessing import OneHotEncoder
```

Remove the old line that encodes the full dataset:

</> Python

```
x = pd.get_dummies(x, columns=textColumns, dtype=int)
```

This keeps the encoder from learning categories from the test set. `handle_unknown="ignore"` lets it transform a test row even if it contains a category not seen during training.

One limitation: this version does not impute missing values or scale numeric columns. If your data has missing values, add an imputer; scaling numeric values may also help logistic regression. A

`ColumnTransformer` inside a `scikit-learn Pipeline` can handle both while ensuring preprocessing is fit only on training data.